

《操作系统》 Lab2

练习1：理解first-fit连续物理内存分配算法

原理阐述：

first-fit算法：连续物理内存分配算法的一种，将空闲内存块按照地址从小到大的方式连起来，具体实现时使用了双向链表的方式。当分配内存时，从链表头开始向后找，这意味着从低地址向高地址查找，一旦找到可以满足要求的内存块，即将该内存块分配出去即可。

代码分析：

整体思路：

首先，我们先从宏观的角度来对问题进行分析，再从整段代码的分配的算法和逻辑进行说明。

该内存管理器使用一个双向循环链表来维护所有空闲的、连续的页块，并按这些空闲页块在物理内存中的起始地址升序进行排列。每个空闲页块的起始页的字段存储了该块的大小。

1.内存初始化

在系统启动时，将一块或多块连续的、可用的物理页块添加到空闲链表。

2.内存分配

遍历空闲链表，找到第一个大小满足请求的空闲页块。

如果找到大小符合要求的空闲页块，就将该页块进行分割，通过分配请求所需的大小n页，如果还有剩余的页，则将剩余的部分作为一个新的、更小的空闲页块保留在链表中。

3.内存释放

将归还的页块标记为空闲，并将其插入到空闲链表中的正确位置，其插入的思想仍然为升序的顺序进行插入。

在插入之后，会检查并合并该页块与它的物理地址上相邻的前一个和/或后一个空闲页块，以减少外部碎片。

具体代码实现

default_init

代码块

```
1 static void
```

```

2  default_init(void) {
3      list_init(&free_list);
4      nr_free = 0;
5  }

```

首先，通过定义一个静态函数default_init来初始化空闲内存块的链表和计数器。其中的 `list_init(&free_list);` 是一个链表初始化函数，其定义在链表操作库中。 `nr_free=0;` 其中变量nr_free用来记录当前空闲内存块的数量，在初始化时将其设为0，表示此时系统还没有可分配的空闲内存块。

default_init_memmap

代码块

```

1  static void
2  default_init_memmap(struct Page *base, size_t n) {
3      assert(n > 0);
4      struct Page *p = base;
5      for (; p != base + n; p++) {
6          assert(PageReserved(p));
7          p->flags = p->property = 0;
8          set_page_ref(p, 0);
9      }
10     base->property = n;
11     SetPageProperty(base);
12     nr_free += n;
13     if (list_empty(&free_list)) {
14         list_add(&free_list, &(base->page_link));
15     } else {
16         list_entry_t* le = &free_list;
17         while ((le = list_next(le)) != &free_list) {
18             struct Page* page = le2page(le, page_link);
19             if (base < page) {
20                 list_add_before(le, &(base->page_link));
21                 break;
22             } else if (list_next(le) == &free_list) {
23                 list_add(le, &(base->page_link));
24             }
25         }
26     }
27 }

```

这一段代码的目的是把从base开始的连续n个struct Page初始化为一个可分配的“空闲块”，并把该块挂入全局的空闲链表free_list中。同时更新空闲页计数nr_free。这是简单物理内存分配器中将一段

页变成“可用”的典型初始化函数。

- `assert(n>0);`

确保传入的页数 n 至少为1，作为参数前置条件检查。

- `struct Page *p = base;`

指针 p 用于遍历以 $base$ 为起点的 $Page$ 数组。

代码块

```
1  for (; p != base + n; p++) {
2      assert(PageReserved(p));
3      p->flags = p->property = 0;
4      set_page_ref(p, 0);
5  }
```

遍历 $base$ 到 $base+n-1$ 的每个结构体，来判断每个待初始化的 $Page$ 是否是被标记为“保留”的页面，如果不是保留页面，程序会触发断言错误并终止，如果是保留页面，然后会清除页面标志和属性，设置页面引用计数为0，将其转换为可用的空闲页面。通过上述操作来确保只对原本就是保留页面的内存区域进行初始化，以及确保内存管理系统的安全性和一致性。

`base->property = n;` 在本段连续内存的首页上设置 $property=n$ ，表示从 $base$ 开始的连续自由页块长度是 n 。且只有块首才需要存放 $property$ ，用于合并、分配时快速知道块大小。

`SetPageProperty(base);` 设置首页的“ $property$ ”标志，表明这个页是一个空闲块的头，这样其他代码可以通过检查 $PageProperty(p)$ 来判断某页是不是一个块头。`nr_free+=n;` 全局可用页计数 nr_free 增加 n 。

代码块

```
1  if (list_empty(&free_list)) {
2      list_add(&free_list, &(base->page_link));
3  }
```

如果当前空闲链表是空的，就直接把 $base$ 的 $page_link$ 加入链表。否则进入 $else$ 分支，把新块插入到 $free_list$ 中合适的位置，也就是下面这一段代码的逻辑。

代码块

```
1  else {
2      list_entry_t* le = &free_list;
3      while ((le = list_next(le)) != &free_list) {
4          struct Page* page = le2page(le, page_link);
5          if (base < page) {
6              list_add_before(le, &(base->page_link));
7              break;
8          }
9      }
```

```

8         } else if (list_next(le) == &free_list) {
9             list_add(le, &(base->page_link));
10        }
11    }
12 }

```

这一段代码是在有序列表中插入新节点的经典算法，其整体的逻辑为将一个新空闲内存块按照地址从低到高的顺序，插入到已排序的空闲链表中。首先从头节点的下一个节点开始遍历，直到再次遇到头节点为止，也就是遍历整个链表。接下来在循环体中，通过获取页面结构体，当新块的地址小于当前遍历块的地址时，将新块放到当前块的前面，插入完成之后，退出循环；如果新块的地址比所有现有块都大，则将新块插入到当前节点之后，即链表末尾。

default_alloc_pages

代码块

```

1  static struct Page *
2  default_alloc_pages(size_t n) {
3      assert(n > 0);
4      if (n > nr_free) {
5          return NULL;
6      }
7      struct Page *page = NULL;
8      list_entry_t *le = &free_list;
9      while ((le = list_next(le)) != &free_list) {
10         struct Page *p = le2page(le, page_link);
11         if (p->property >= n) {
12             page = p;
13             break;
14         }
15     }
16     if (page != NULL) {
17         list_entry_t* prev = list_prev(&(page->page_link));
18         list_del(&(page->page_link));
19         if (page->property > n) {
20             struct Page *p = page + n;
21             p->property = page->property - n;
22             SetPageProperty(p);
23             list_add(prev, &(p->page_link));
24         }
25         nr_free -= n;
26         ClearPageProperty(page);
27     }
28     return page;
29 }

```

这一段代码主要是用来**分配空闲内存块**。通过遍历空闲链表，找到第一个大小足够容纳请求的连续空闲内存块，然后进行分配，如有剩余则分割并放回空闲链表。

代码块

```
1  assert(n > 0);
2      if (n > nr_free) {
3          return NULL;
4      }
```

首先是可行性分析，检查请求的页面数 n 是否合法，也就是大于0，同时，检查系统中是否有足够的空闲页面，如果不够，则直接返回`NULL`表示分配失败。

代码块

```
1  struct Page *page = NULL;
2      list_entry_t *le = &free_list;
3      while ((le = list_next(le)) != &free_list) {
4          struct Page *p = le2page(le, page_link);
5          if (p->property >= n) {
6              page = p;
7              break;
8          }
9      }
```

在这一步，通过初始化`page`为`NULL`，表示尚未找到合适块，然后遍历空闲链表`free_list`，对每个空闲块，检查其`property`，也就是连续空闲页面数，是否大于等于请求的 n ，根据`first fit`的逻辑可知，我们找到第一个满足条件的块就立即停止，如果遍历完都没找到，`page`保持`NULL`。

代码块

```
1  if (page != NULL) {
2      list_entry_t* prev = list_prev(&(page->page_link));
3      list_del(&(page->page_link));
4      if (page->property > n) {
5          struct Page *p = page + n;
6          p->property = page->property - n;
7          SetPageProperty(p);
8          list_add(prev, &(p->page_link));
9      }
10     nr_free -= n;
11     ClearPageProperty(page);
12 }
```

如果`page`不是`NULL`，也就是找到了合适的块，首先记录前驱节点，然后将其从链表中移除，之后这个块就不再属于空闲链表，之后就进入了这一段代码的关键步骤，对块进行分割，如果找到的块比请求的`n`大，也就是说有剩余，则需要计算剩余部分的起始页面，在原有的`page`的基础上再加上`n`，来重新调整剩余部分的起始页面，同时，设置剩余块的大小，标记剩余块为空闲状态，同时再将剩余块插回原位置，来保持链表有序。更新系统的状态，更新全局空闲页面计数，清除分配块的“空闲属性”标记，返回分配块的起始页面指针。

default_free_pages

代码块

```
1  static void
2  default_free_pages(struct Page *base, size_t n) {
3      assert(n > 0);
4      struct Page *p = base;
5      for (; p != base + n; p++) {
6          assert(!PageReserved(p) && !PageProperty(p));
7          p->flags = 0;
8          set_page_ref(p, 0);
9      }
10     base->property = n;
11     SetPageProperty(base);
12     nr_free += n;
13
14     if (list_empty(&free_list)) {
15         list_add(&free_list, &(base->page_link));
16     } else {
17         list_entry_t* le = &free_list;
18         while ((le = list_next(le)) != &free_list) {
19             struct Page* page = le2page(le, page_link);
20             if (base < page) {
21                 list_add_before(le, &(base->page_link));
22                 break;
23             } else if (list_next(le) == &free_list) {
24                 list_add(le, &(base->page_link));
25             }
26         }
27     }
28
29     list_entry_t* le = list_prev(&(base->page_link));
30     if (le != &free_list) {
31         p = le2page(le, page_link);
32         if (p + p->property == base) {
33             p->property += base->property;
34             ClearPageProperty(base);
35             list_del(&(base->page_link));
36             base = p;
```

```

37     }
38 }
39
40 le = list_next(&(base->page_link));
41 if (le != &free_list) {
42     p = le2page(le, page_link);
43     if (base + base->property == p) {
44         base->property += p->property;
45         ClearPageProperty(p);
46         list_del(&(p->page_link));
47     }
48 }
49 }

```

这段代码整体是用来进行**内存的释放与空闲块的合并**。通过释放已分配的物理页面，并将它们与相邻的空闲块合并，以减少内存碎片。

代码块

```

1  for (; p != base + n; p++) {
2      assert(!PageReserved(p) && !PageProperty(p));
3      p->flags = 0;
4      set_page_ref(p, 0);
5  }

```

首先，进行页面的初始化，先确保要释放的页面既不是保留页面也不是空闲页面，也就是说确认其为已分配页面，然后清除页面标志，引用计数归零。从而将这些页面恢复到“干净状态”，准备作为空闲页面重用。

代码块

```

1  base->property = n;
2  SetPageProperty(base);
3  nr_free += n;

```

将释放的 n 个页面标记为一个连续空闲块。 $base$ 作为该块的头页面，记录块大小，然后标记标记 $base$ 为空闲状态，最后再更新全局空闲页面计数器。

代码块

```

1  if (list_empty(&free_list)) {
2      list_add(&free_list, &(base->page_link));
3  } else {
4      list_entry_t* le = &free_list;

```

```

5         while ((le = list_next(le)) != &free_list) {
6             struct Page* page = le2page(le, page_link);
7             if (base < page) {
8                 list_add_before(le, &(base->page_link));
9                 break;
10            } else if (list_next(le) == &free_list) {
11                list_add(le, &(base->page_link));
12            }
13        }
14    }

```

然后是插入有序空闲链表，如果空闲链表为空，则直接插入，否则的话按照地址从低到高的顺序找到正确位置插入，以此来保持链表有序，便于后续的操作。

代码块

```

1  list_entry_t* le = list_prev(&(base->page_link));
2  if (le != &free_list) {
3      p = le2page(le, page_link);
4      if (p + p->property == base) {
5          p->property += base->property;
6          ClearPageProperty(base);
7          list_del(&(base->page_link));
8          base = p;
9      }
10 }
11 le = list_next(&(base->page_link));
12 if (le != &free_list) {
13     p = le2page(le, page_link);
14     if (base + base->property == p) {
15         base->property += p->property;
16         ClearPageProperty(p);
17         list_del(&(p->page_link));
18     }
19 }

```

这一部分是函数的核心部分：**空闲块的合并**。分为前向合并和后向合并。前向合并的逻辑为先获取前一个块，检查是否是地址连续，如果地址连续，也就是满足条件： $p+p->property==base$ ，则合并大小，清除当前块的标记，从链表移除当前块， $base$ 指向前一个块，也就是合并之后的块。

然后进行后向合并的逻辑判断，先获取后一个块，检查地址是否连续，如果地址连续，则合并大小，然后清除后块的标记，从链表中移除后块。

default_free_pages


```

1 代码块 static void
2  default_free_pages(struct Page *base, size_t n) {
3     assert(n > 0);
4     struct Page *p = base;
5     for (; p != base + n; p++) {
6         assert(!PageReserved(p) && !PageProperty(p));
7         p->flags = 0;
8         set_page_ref(p, 0);
9     }
10    base->property = n;
11    SetPageProperty(base);
12    nr_free += n;
13
14    if (list_empty(&free_list)) {
15        list_add(&free_list, &(base->page_link));
16    } else {
17        list_entry_t* le = &free_list;
18        while ((le = list_next(le)) != &free_list) {
19            struct Page* page = le2page(le, page_link);
20            if (base < page) {
21                list_add_before(le, &(base->page_link));
22                break;
23            } else if (list_next(le) == &free_list) {
24                list_add(le, &(base->page_link));
25            }
26        }
27    }
28
29    list_entry_t* le = list_prev(&(base->page_link));
30    if (le != &free_list) {
31        p = le2page(le, page_link);
32        if (p + p->property == base) {
33            p->property += base->property;
34            ClearPageProperty(base);
35            list_del(&(base->page_link));
36            base = p;
37        }
38    }
39
40    le = list_next(&(base->page_link));
41    if (le != &free_list) {
42        p = le2page(le, page_link);
43        if (base + base->property == p) {
44            base->property += p->property;
45            ClearPageProperty(p);
46            list_del(&(p->page_link));
47        }

```

```
48     }
49 }
```

这一段函数用于释放内存块。将释放的内存块按照顺序插入空闲内存块的链表中，并合并与之相邻且连续的空闲内存块。

首先，如果该页面的保留属性和页面数量属性均不为初始值了，我们就重置页面的对应属性，将引用设置定义为0。然后对应的更新空闲块数的数量。将页面添加到空闲块列表中，同时尝试合并相邻的空闲块。如果释放的页面与前一个页面或后一个页面相邻，会尝试将它们合并为一个更大的空闲块。

default_nr_free_pages

代码块

```
1  static size_t
2  default_nr_free_pages(void) {
3      return nr_free;
4  }
```

这个一段代码是用来**获取系统当前空闲数量的函数**。类型为 `size_t`，表示空闲页面的数量。其直接返回全局变量 `nr_free` 的值。

basic_check

代码块

```
1  static void
2  basic_check(void) {
3      struct Page *p0, *p1, *p2;
4      p0 = p1 = p2 = NULL;
5      assert((p0 = alloc_page()) != NULL);
6      assert((p1 = alloc_page()) != NULL);
7      assert((p2 = alloc_page()) != NULL);
8
9      assert(p0 != p1 && p0 != p2 && p1 != p2);
10     assert(page_ref(p0) == 0 && page_ref(p1) == 0 && page_ref(p2) == 0);
11
12     assert(page2pa(p0) < npage * PGSIZE);
13     assert(page2pa(p1) < npage * PGSIZE);
14     assert(page2pa(p2) < npage * PGSIZE);
15
16     list_entry_t free_list_store = free_list;
17     list_init(&free_list);
18     assert(list_empty(&free_list));
19
20     unsigned int nr_free_store = nr_free;
```

```

21     nr_free = 0;
22
23     assert(alloc_page() == NULL);
24
25     free_page(p0);
26     free_page(p1);
27     free_page(p2);
28     assert(nr_free == 3);
29
30     assert((p0 = alloc_page()) != NULL);
31     assert((p1 = alloc_page()) != NULL);
32     assert((p2 = alloc_page()) != NULL);
33
34     assert(alloc_page() == NULL);
35
36     free_page(p0);
37     assert(!list_empty(&free_list));
38
39     struct Page *p;
40     assert((p = alloc_page()) == p0);
41     assert(alloc_page() == NULL);
42
43     assert(nr_free == 0);
44     free_list = free_list_store;
45     nr_free = nr_free_store;
46
47     free_page(p);
48     free_page(p1);
49     free_page(p2);
50 }

```

总的来说，这一段代码是一个**内存管理基础功能的单元测试函数**，用于验证页面分配和释放的核心逻辑是否正确工作。这段代码分别验证单个页面的分配、释放、状态管理是否正常工作。是对上述的函数的功能的验证。

default_check

代码块

```

1  static void
2  default_check(void) {
3      int count = 0, total = 0;
4      list_entry_t *le = &free_list;
5      while ((le = list_next(le)) != &free_list) {
6          struct Page *p = le2page(le, page_link);
7          assert(PageProperty(p));

```

```
8         count ++, total += p->property;
9     }
10    assert(total == nr_free_pages());
11
12    basic_check();
13
14    struct Page *p0 = alloc_pages(5), *p1, *p2;
15    assert(p0 != NULL);
16    assert(!PageProperty(p0));
17
18    list_entry_t free_list_store = free_list;
19    list_init(&free_list);
20    assert(list_empty(&free_list));
21    assert(alloc_page() == NULL);
22
23    unsigned int nr_free_store = nr_free;
24    nr_free = 0;
25
26    free_pages(p0 + 2, 3);
27    assert(alloc_pages(4) == NULL);
28    assert(PageProperty(p0 + 2) && p0[2].property == 3);
29    assert((p1 = alloc_pages(3)) != NULL);
30    assert(alloc_page() == NULL);
31    assert(p0 + 2 == p1);
32
33    p2 = p0 + 1;
34    free_page(p0);
35    free_pages(p1, 3);
36    assert(PageProperty(p0) && p0->property == 1);
37    assert(PageProperty(p1) && p1->property == 3);
38
39    assert((p0 = alloc_page()) == p2 - 1);
40    free_page(p0);
41    assert((p0 = alloc_pages(2)) == p2 + 1);
42
43    free_pages(p0, 2);
44    free_page(p2);
45
46    assert((p0 = alloc_pages(5)) != NULL);
47    assert(alloc_page() == NULL);
48
49    assert(nr_free == 0);
50    nr_free = nr_free_store;
51
52    free_list = free_list_store;
53    free_pages(p0, 5);
54
```

```

55     le = &free_list;
56     while ((le = list_next(le)) != &free_list) {
57         struct Page *p = le2page(le, page_link);
58         count --, total -= p->property;
59     }
60     assert(count == 0);
61     assert(total == 0);
62 }

```

这一段函数实现了比上述的 `basic_check()` 函数更加全面的**内存管理算法验证测试**，比之前的 `basici_check()` 更加复杂和完整。测试了部分页面的释放和精确分配，碎片化内存管理，以及完全分配与合并。

结构体 `pmm_manager default_pmm_manager`

代码块

```

1  const struct pmm_manager default_pmm_manager = {
2      .name = "default_pmm_manager",
3      .init = default_init,
4      .init_memmap = default_init_memmap,
5      .alloc_pages = default_alloc_pages,
6      .free_pages = default_free_pages,
7      .nr_free_pages = default_nr_free_pages,
8      .check = default_check,
9  };

```

这一部分是**完整的内存管理模块接口的实现**，采用了面向对象的设计模式。以下是对这个结构体的各个成员的解释：

- `.name = "default_pmm_manager"`：用于标识这个内存管理器的名称。
- `.init = default_init`：函数指针，用于初始化内存管理器的某些状态。
- `.init_memmap = default_init_memmap`：函数指针，用于设置内存页面的初始状态。
- `.alloc_pages = default_alloc_pages`：函数指针，指向一个用于分配页面的函数。
- `.free_pages = default_free_pages`：函数指针，指向一个用于释放页面的函数。
- `.nr_free_pages = default_nr_free_pages`：函数指针，指向一个用于获取空闲页面数量的函数。
- `.check = default_check`：函数指针，用于分配情况的检查。

改进空间：

物理内存视图 (一个格子代表1个页, 大小为4KB):

```

2   | P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10| P11| ...
3   +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

按阶 (order) 管理空闲块: 阶 `k` 表示大小为 `2^k` 页的块。

`MAX_ORDER` 为最大的阶。

以 `order = 2` (每块4页) 为例, 举一个简单的例子:

页号	页号的二进制表示	所属块	块号	伙伴块号
0	0000	[0,1,2,3]	0	1
4	0100	[4,5,6,7]	1	0
8	1000	[8,9,10,11]	2	3
12	1100	[12,13,14,15]	3	2

分配时, 找到最小 `order` 使得 `2^order >= n`, 分配该阶 (可能通过从更高阶拆分得到)。

释放时, 先把 `n` 转成对应 `order`, 释放并尝试与伙伴合并到更高阶。

2. 关键数据结构

- `buddy_system_t buddy`: 核心结构体
 - `list_entry_t free_list[MAX_ORDER + 1]`: 每个阶一个链表头, 存放各阶空闲块头 (块头由 `struct Page` 承担)。
 - `unsigned int nr_free[MAX_ORDER + 1]`: 每阶的空闲块数量, 用于统计和检查是否有空闲块。

`order` 是从0开始取值, 故数组大小应该为 `MAX_ORDER + 1`。

- `static struct Page *buddy_base` 和 `static size_t buddy_npages`
 - `buddy_base` 指向 `pmm->init_memmap` 传入的 `base` (即 `&pages[first_free_index]`), 所有对页的索引与伙伴计算都以它为基准。

3. 辅助函数

- `page_index`: 计算相对页索引, 便于伙伴计算。
- `in_range`: 边界检查, 防止越界。
- `buddy_of`: 计算伙伴页面, 确保在范围内否则返回 `NULL`。

核心逻辑: `buddy_idx = idx ^ (1 << order)`

- `1 << order` 对应的二进制位是当前块的“层级位”；
- XOR（异或）操作相当于“翻转”这个层级位；
- 翻转该位，意味着从当前块跳到它的“另一半”——伙伴块。

例如，在上面的表格中，`order = 2`。对于 `idx = 12` 的块，计算它的伙伴块 `buddy_idx = idx ^ (1 << order) = 12 ^ (1 << 2) = 12 ^ 4 = 8`。

- `buddy_nr_free_pages`：空闲页数量统计。将每一个order的空闲页累加即可。

4. 全局初始化： `buddy_init(void)`

目标：清空所有链表和计数器，便于内核启动时调用。

5. 初始化空闲区： `buddy_init_memmap(base, n)`

目标：把从 `base` 开始的 `n` 个物理页划分成尽可能大的 2^k 连续页块，并把这些块加入各阶（order）空闲链表free lists中，同时保证每个放入的块头满足对齐要求（**每个块的起始地址必须是块大小的倍数，即按 2^{order} 对齐**）。

对齐才能通过异或运算 `(idx ^ (1 << order))` 快速计算伙伴块。

实现要点：

- 记录 `buddy_base = base; buddy_npages = n;`
- 对所有新加入的页先清除 `Reserved / Property` 标志（`ClearPageReserved / ClearPageProperty`），并让 `property=0`。
- 使用**贪心分割**：从当前位置 `idx` 开始，找出最大的 `order` 使得 `(1<<order) <= remain`（剩余页数），并把该块放入该阶链表。这里假定 `base` 本身就按页对齐并且我们从连续区段0开始编号，从而 `idx` 自然满足 `idx % (1<<order) == 0`（即刚好对齐）。

6. 分配流程： `buddy_alloc_pages(size_t n)`

目标：分配至少 `n` 个连续的空闲页，返回块头 `struct Page*`（实质上会分配连续 `1<<order` 页）。

步骤与设计理由：

1. 计算 `order = ceil_log2(n)`（最小的 `order` 使 `1<<order >= n`）。
2. 从 `order` 开始向上查找第一个非空free_list（`cur`）。如果到 `MAX_ORDER` 都没有，分配失败返回NULL。
3. 从 `free_list[cur]` 取一个块头，把它从链表中移除（`list_del`）并更新 `buddy.nr_free[cur]`（维护计数）。
4. 如果 `cur == order`，则无需拆分，直接分配。
如果 `cur > order`，则不断拆分：

把当前阶为 `cur` 块拆成两个阶为 `cur-1` 的块：左侧保持作为待继续拆分/返回的块，右侧块头为 `page + (1 << (cur-1))`（即伙伴页地址），将右侧加入 `free_list[cur-1]` 并 `buddy.nr_free[cur-1]++`。

重复直到 `cur == order`。

5. 在最终返回前做状态标记：清 `PageProperty`、把 `page->property = order`。

7. 释放流程与合并： `buddy_free_pages(base, n)`

目标：把 `base` 指向的一段连续页（`n` 页）释放回伙伴系统，并尽可能向上合并成更大阶的块。

步骤与设计理由：

1. 通过第1个 `while` 循环计算 `order = ceil_log2(n)`（和分配一样，得到最小的 `order` 使 `1<<order >= n`）。
2. 从当前块头 `page=base` 开始，进入第2个 `while` 循环：
 - 以下3个条件任意一个条件成立，就停止合并，退出循环：
 - `buddy_page == NULL` 表示计算出来的“伙伴页”不在有效的内存管理范围之内。
 - `!PageProperty(buddy_page)` 表示伙伴页不在空闲状态，即被分配（正在使用）。
 - `buddy_page->property != order` 表示伙伴块的阶次与当前块的阶次不同。
 - 否则就要进行合并：把伙伴从 `free_list[order]` 删除（`list_del`），`buddy.nr_free[order]--`；然后把 `page` 指向合并后的新块头（`min(page, buddy_page)`）；`order++` 阶加1；继续向上尝试合并。
3. 把合并后的块加入 `free_list[order]`（此时`order`已经完成了加1），`SetPageProperty(page)`，`page->property = order`，并更新 `buddy.nr_free[order]++`。

代码实现

```
buddy_pmm.h

1  #ifndef __KERN_MM_BUDDY_PMM_H__
2  #define __KERN_MM_BUDDY_PMM_H__
3
4  #include <pmm.h>
5  #include <list.h>
6  #include <memlayout.h>
7
8  #define MAX_ORDER 10 // 最大阶数 (1 << 10) 页 = 4MB
9
10 typedef struct
11 {
12     list_entry_t free_list[MAX_ORDER + 1]; // 各阶空闲块链表
```

```
13     unsigned int nr_free[MAX_ORDER + 1];    // 各阶空闲块数量
14 } buddy_system_t;
15
16 extern const struct pmm_manager buddy_pmm_manager;
17
18 #endif /* !__KERN_MM_BUDDY_PMM_H__ */
```

buddy_pmm.c

```
1  #include <buddy_pmm.h>
2  #include <assert.h>
3  #include <string.h>
4  #include <stdio.h>
5
6  static buddy_system_t buddy;
7  static struct Page *buddy_base;
8  static size_t buddy_npages;
9
10 // 获取页索引
11 static inline size_t page_index(struct Page *page){
12     return page - buddy_base;
13 }
14
15 // 判断是否在范围内
16 static inline int in_range(struct Page *page){
17     return page >= buddy_base && page < buddy_base + buddy_npages;
18 }
19
20 // 获取伙伴页
21 static inline struct Page *buddy_of(struct Page *page, int order){
22     size_t idx = page_index(page);
23     size_t buddy_idx = idx ^ (1 << order);
24     if (buddy_idx >= buddy_npages)
25         return NULL;
26     return buddy_base + buddy_idx;
27 }
28
29 // 初始化
30 static void buddy_init(void){
31     for (int i = 0; i <= MAX_ORDER; i++)
32     {
33         list_init(&buddy.free_list[i]);
34         buddy.nr_free[i] = 0;
35     }
36     cprintf("buddy_pmm initialized.\n");
37 }
```

```
38
39 // 初始化空闲区
40 static void buddy_init_memmap(struct Page *base, size_t n){
41     assert(n > 0);
42     buddy_base = base;
43     buddy_npages = n;
44
45     for (size_t i = 0; i < n; i++){
46         ClearPageReserved(base + i);
47         ClearPageProperty(base + i);
48         base[i].property = 0;
49     }
50
51     // 尽可能按最大块划分
52     size_t idx = 0;
53     while (n > 0){
54         int order = MAX_ORDER;
55         while ((1 << order) > n)
56             order--;
57         struct Page *p = base + idx;
58         SetPageProperty(p);
59         p->property = order;
60         list_add(&buddy.free_list[order], &(p->page_link));
61         buddy.nr_free[order]++;
62         idx += (1 << order);
63         n -= (1 << order);
64     }
65 }
66
67 // 分配页
68 static struct Page *buddy_alloc_pages(size_t n){
69     if (n == 0)
70         return NULL;
71
72     int order = 0;
73     while ((1U << order) < n && order <= MAX_ORDER)
74         order++;
75     if (order > MAX_ORDER)
76         return NULL;
77
78     int cur = order;
79     while (cur <= MAX_ORDER && list_empty(&buddy.free_list[cur]))
80         cur++;
81
82     if (cur > MAX_ORDER)
83         return NULL; // 没有足够大的块
84 }
```

```

85
86     struct Page *page;
87     list_entry_t *le = list_next(&buddy.free_list[cur]);
88     page = le2page(le, page_link);
89     list_del(le);
90     buddy.nr_free[cur]--;
91
92     // 从更高阶分裂下来, 如果不需要寻找更高阶则不进入循环
93     while (cur > order){
94         cur--;
95         struct Page *buddy_page = page + (1 << cur); // 计算伙伴页地址
96         SetPageProperty(buddy_page);
97         buddy_page->property = cur;
98         list_add(&buddy.free_list[cur], &(buddy_page->page_link)); // 伙伴页并入
    低一阶链表
99         buddy.nr_free[cur]++;
100     }
101
102     ClearPageProperty(page);
103     page->property = order;
104     return page;
105 }
106
107 // 释放页
108 static void buddy_free_pages(struct Page *base, size_t n){
109     assert(in_range(base));
110
111     int order = 0;
112     while ((1U << order) < n && order <= MAX_ORDER)
113         order++;
114     assert(order <= MAX_ORDER);
115
116     struct Page *page = base;
117     while (order < MAX_ORDER){
118         struct Page *buddy_page = buddy_of(page, order);
119         if (buddy_page == NULL || !PageProperty(buddy_page) || buddy_page->property != order)
120             break;
121         // 从链表中移除伙伴
122         list_del(&(buddy_page->page_link));
123         buddy.nr_free[order]--;
124         // 合并
125         if (buddy_page < page)
126             page = buddy_page;
127         order++;
128     }
129

```

```
130     SetPageProperty(page);
131     page->property = order;
132     list_add(&buddy.free_list[order], &(page->page_link));
133     buddy.nr_free[order]++;
134 }
135
136 // 空闲页数量统计
137 static size_t buddy_nr_free_pages(void){
138     size_t sum = 0;
139     for (int i = 0; i <= MAX_ORDER; i++)
140         sum += buddy.nr_free[i] * (1 << i);
141     return sum;
142 }
143
144 // 自检函数
145 static void buddy_check(void){
146     cprintf("===== buddy_check() START =====\n");
147
148     struct Page *p1, *p2, *p3, *p4, *p5;
149
150     // 基本分配测试
151     p1 = buddy_alloc_pages(1);
152     p2 = buddy_alloc_pages(2);
153     assert(p1 && p2);
154     cprintf("[OK] Basic alloc (1, 2 pages)\n");
155
156     // 非2的幂次分配 (3页应该分配为4页块)
157     p3 = buddy_alloc_pages(3);
158     assert(p3);
159     cprintf("[OK] Non-power-of-two alloc (3 pages => 4)\n");
160
161     // 边界测试: 申请超过总页数
162     struct Page *fail = buddy_alloc_pages(buddy_npages + 1);
163     assert(fail == NULL);
164     cprintf("[OK] Oversized alloc rejected\n");
165
166     // 释放部分块, 测试是否可重用
167     free_pages(p1, 1);
168     free_pages(p2, 2);
169     cprintf("[OK] Partial free\n");
170
171     // 交错分配与释放
172     p4 = buddy_alloc_pages(5); // 分配5页 (应该拿8页块)
173     assert(p4);
174     p5 = buddy_alloc_pages(1);
175     assert(p5);
176     free_pages(p4, 5);
```

```
177     free_pages(p3, 3);
178     free_pages(p5, 1);
179     cprintf("[OK] Interleaved alloc/free\n");
180
181     // 检查完全恢复
182     size_t max_npages = 1;
183     while(max_npages << 1 <= buddy_npages)
184         max_npages <<= 1;
185     struct Page *big = buddy_alloc_pages(max_npages);
186     assert(big);
187     free_pages(big, max_npages);
188     cprintf("[OK] Full merge check passed\n");
189
190     cprintf("===== buddy_check_full() PASSED =====\n");
191 }
192
193 const struct pmm_manager buddy_pmm_manager = {
194     .name = "buddy_pmm_manager",
195     .init = buddy_init,
196     .init_memmap = buddy_init_memmap,
197     .alloc_pages = buddy_alloc_pages,
198     .free_pages = buddy_free_pages,
199     .nr_free_pages = buddy_nr_free_pages,
200     .check = buddy_check,
201 };
```

自检测试

自检函数 `buddy_check()` 将测试以下几种情况：

测试类型	说明
基本分配	申请 1、2、4页等标准块
非2的幂次分配	比如申请3页、5页，测试伙伴系统自动“向上取整”行为
多次分配 + 释放	检查是否出现空洞、链表破坏
交错分配与释放	检查合并逻辑是否稳定
边界测试	尝试分配超出总页数的请求
完整回收验证	最后尝试分配整个内存，验证系统完全恢复

自检函数 `buddy_check()` 的完整代码详见“代码实现”部分。执行 `make qemu`，输出如下内容，说明通过了自检，编写的伙伴系统运行正确。

```
Special kernel symbols:
  entry 0xffffffffc02000d8 (virtual)
  etext 0xffffffffc02011aa (virtual)
  edata 0xffffffffc0205018 (virtual)
  end    0xffffffffc02051b0 (virtual)
Kernel executable memory footprint: 21KB
memory management: buddy_pmm_manager
buddy_pmm initialized.
physical memory map:
  memory: 0x0000000008000000, [0x0000000008000000, 0x00000000087fffffff].
===== buddy_check() START =====
[OK] Basic alloc (1, 2 pages)
[OK] Non-power-of-two alloc (3 pages => 4)
[OK] Oversized alloc rejected
[OK] Partial free
[OK] Interleaved alloc/free
[OK] Full merge check passed
===== buddy_check_full() PASSED =====
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0204000
satp physical address: 0x00000000080204000
```

扩展练习Challenge：任意大小的内存单元slub分配算法

设计文档

1. 基本思想

SLUB 是一种高效的小对象内存分配机制。它将内存页（Page）划分为若干个对象池（Slab），每个 Slab 存放相同大小的对象。系统为不同对象大小维护独立的 Cache，每个 Cache 包含若干个 Slab，并按使用状态分为 `Full`、`Partial`、`Free` 三类链表。

分配时，从 `Partial` 或 `Free` 的 Slab 中取出一个对象；释放时，将对象插回 Slab 的空闲链表（freelist），并根据使用计数调整 Slab 的归属。当一个 Slab 中的所有对象都释放后，对应页即可归还系统。

这种方式通过**对象大小分级**、**页内对象池管理与空闲链表快速分配**，显著降低碎片，提高了小对象分配的效率与缓存命中率。

2. 关键数据结构(slub.h)

- `kmem_cache_t`：描述某一固定对象尺寸的缓存，维护三个 slab 链表（full/partial/free）、对象大小 `obj_size`、每 slab 对象数量 `total`（由 slab 计算）、以及全局缓存链表节点 `cache_link`。
- `slab_t`：位于页起始处的 slab 元数据，包含指向所属 cache 的指针、空闲对象链表头 `freelist`、在用计数 `inuse`、总对象数 `total`、以及链接到 cache 的 `slab_link`。

- le2slab 宏：从双向链表节点还原到 slab 指针，便于链表与结构体互转。

核心结构在 `kern/mm/slub.h` 中定义。关键片段如下：

代码块

```
1 // kern/mm/slub.h
2 typedef struct kmemCache{
3     list_entry_t slabFull;    // 满载slab
4     list_entry_t slabPartial; // 部分占用slab
5     list_entry_t slabFree;    // 空闲slab
6     size_t objSize;           // 对象大小
7     size_t slabSize;          // slab大小，简化为一页
8     list_entry_t cacheLink;   // 加入全局cache链
9 } kmem_cache_t;
10
11 typedef struct slabTag{
12     kmem_cache_t * cachep;    // 反向指向所属cache
13     void * freelist;           // 页内空闲对象单链表
14     int inuse;                 // 已分配对象数
15     int total;                 // 总对象数
16     list_entry_t slabLink;    // 链接到cache的slab链
17 } slab_t;
18
19 #define le2slab(le, member) to_struct((le), slab_t, member)
```

3. 辅助函数

对象从 slab freelist 分配的核心在 `slab_alloc_object`，代码如下（`kern/mm/slub.c`）：

代码块

```
1 static void *slab_alloc_object(slab_t *slabp) {
2     if (slabp->freelist == NULL) return NULL;
3     void *obj = slabp->freelist;
4     slabp->freelist = *((void **)obj); // 下一空闲
5     return obj;
6 }
```

说明：

- 错误模式：`freelist==NULL` 时返回 NULL，由上层决定是否 grow。
- 不进行对象构造（ctor）与清零（为性能考虑）；调用方如需置零可自行 `memset`。

- 单链表头插，常数时间；避免跨 cache/sizes 的对象混用。

4. 全局初始化

在 `pmm_init()` 之后调用，创建 `kmalloc` 用的预定义 cache (8~4096B)：

代码块

```
1 void slubInit() {
2     list_init(&cacheChain);
3     kmallocCaches[0] = kmemCacheCreate(8);
4     kmallocCaches[1] = kmemCacheCreate(16);
5     // ... 32, 64, ..., 4096
6     kmallocCaches[9] = kmemCacheCreate(4096);
7     cprintf("slub_init: SLUB allocator initialized\n");
8 }
```

在 `kern/init/init.c` 中集成：

代码块

```
1 // kern/init/init.c
2 #include <slub.h>
3 ...
4 pmm_init();
5 slubInit();
```

5. 初始化空闲区

当缓存没有可用 slab 时，从 PMM 申请一页并格式化。下列片段展示了页到内核 VA 的转换、页内对象数量的计算，以及将对象串成 freelist 的要点：

代码块

```
1 uintptr_t pa = page2pa(page);
2 void *page_va = (void *) (pa + va_pa_offset);
3 size_t available = PGSIZE - sizeof(slab_t);
4 slabp->total = available / cachep->objSize;
5 void *obj = (char *)page_va + sizeof(slab_t);
6 // 将页内对象串成 freelist (步长为 objSize)
```

6. 分配流程

分配的决策顺序是“优先 partial → 其次 free → 否则 grow”。成功取到对象后更新 inuse，若恰好满载则迁移到 full。核心逻辑可以压缩为：

代码块

```
1  if (!list_empty(&cache->slabPartial)) pick_partial();
2  else if (!list_empty(&cache->slabFree)) move_free_to_partial();
3  else slabp = kmemCacheGrow(cache);
4  obj = pop_from_freelist(slabp);
5  if (++slabp->inuse == slabp->total) move_to_full(slabp);
```

7. 释放流程：

释放通过页对齐回溯到所属 slab，将对象头插回 freelist，更新计数后根据阈值在 full/partial/free 之间迁移：

代码块

```
1  slab_t *slabp = (slab_t *)((uintptr_t)objp & ~(PGSIZE - 1));
2  *((void **)objp) = slabp->freelist; slabp->freelist = objp;
3  if (--slabp->inuse == slabp->total - 1) move_full_to_partial(slabp);
4  else if (slabp->inuse == 0) move_partial_to_free(slabp);
```

说明：

- 通过页对齐反推 slab 常数时间完成，避免额外 map；要求 slab 元数据在页首。
- 双条件迁移保障分类不变量：full→partial（释放1个）、partial→free（变为0）。
- 潜在风险：并发环境下需加锁/关中断；对象二次释放需上层保证或加调试魔数。

运行结果

采用指令 `timeout 10 make qemu` 编译运行内核，验证 SLUB 初始化与运行正确性：

- kmemCacheFree(cachep, objp): 释放对象回其所属 slab, 更新 inuse, 并在 full/partial/free 之间迁移。
- kmalloc/kfree: 面向通用调用者的接口。kmalloc 根据 size 选择最小可容纳的预定义 cache, 再调用 kmemCacheAlloc; kfree 通过对象地址定位所属 slab 与 cache, 然后调用 kmemCacheFree。

目前实现的缺陷

- 简化版实现, slab 元数据直接占用页首, 尚未实现更灵活的元数据布局或对象对齐优化。
- 未实现 kmemCacheDestroy 以及 slab 回收至 PMM 的策略 (可在 free 列表过多时回收)。
- 未引入多核/中断上下文并发保护 (生产实现需加锁或禁用中断临界区)。

扩展练习Challenge: 硬件的可用物理内存范围的获取方法

通过查阅资料, 可以使用15h中断, 具体的步骤如下:

1. 将寄存器ax赋值为0E820h
2. 将寄存器ebx初始化为0, 该寄存器的内容会被BIOS修改, 同时, 我们需要保证在内存查询过程中, 该寄存器不会被修改。
3. es:di指向一块足够大的内存地址, BIOS会把有关内存的信息写到这个地址, 内存信息是一种数据结构, 称之为地址范围描述符。
4. ecx寄存器存储es:di所指向的内存大小, 以字节为单位, BIOS最多会填充ecx个字节的数据, 通常情况下, 无论ecx的数值是多少, BIOS都只填充20字节, 有些BIOS直接忽略ecx的值, 总是填充20字节。
5. edx寄存器的值设置为0534D4150h, 这个数值其实对应的是字符组合“SMAP”, 其作用我们可以暂时忽略。

完成上述配置之后, 执行int 15h中断, 中断结果的分析过程如下:

1. 判断CF位, 如果CF位设置为1, 则表示出错
2. eax会被设置为0534D4150h, 也就是字符串SMAP
3. es:di返回地址范围描述符结构指针, 跟输入时相同
4. 如果ebx的值为0, 表明查询结束, 如果不为0, 则继续调用15h获取有关内存的信息。

实验中的重要知识点与 OS 原理中对应的知识点

1.使用 双向循环链表 管理空闲内存块：

涉及到**连续内存分配机制**、**空闲块管理结构**。

1. 实验中通过链表结构来维护物理内存空闲块，是具体实现。
2. OS 原理中讲的是分区分配算法理论，强调“如何管理空闲内存”的思想，如空闲分区表、空闲分区链。
3. 二者的关系是：实验实现了理论中的“空闲分区链”形式；差异在于实验使用的是页为单位，实际 OS 可能还涉及段页式、Buddy 等更复杂机制。

2.First-Fit 算法的实现与代码

也就是First-Fit 连续内存分配算法原理。

1. 原理上：从低地址开始查找第一个足够大的空闲块，然后分配。
2. 实验上：通过遍历空闲链表，从头开始顺序查找，找到第一个满足要求的块并分割。
3. 二者关系：实验是对原理的直接编码实现；差异在于实验用页粒度实现连续页分配，而原理通常是以字节/分区大小描述。

3.Best-Fit 算法的实现

Best-Fit 连续内存分配算法原理。

1. 原理上：从所有空闲块中找出最“合适”的（剩余空间最少但仍够用）的空闲块，减少内部碎片。
2. 实验上：遍历链表，记录符合条件的最小块，再进行分割。
3. 二者关系：实验是原理的直接实现；差异在于实验是以页块为单位实现，OS 课本讲的是抽象分区大小。

4.内存释放与块合并机制

外部碎片管理与空闲块合并

1. 原理上：释放后需合并相邻空闲块，以减少外部碎片，维持空闲块的连续性。
2. 实验中：释放时将块按地址插入到链表正确位置，并检查前后相邻块进行合并。
3. 二者关系：实验很好地还原了理论中的“合并空闲分区”的思想；差异在于实验以页为单位，而理论强调连续分区管理。

5.初始化空闲内存链表

系统启动时内存布局与空闲内存初始化。

1. 原理上：系统启动时，内核需建立内存管理的数据结构，记录可用物理内存区域。
2. 实验中：通过手动设置可用页块，并把它们插入空闲链表。

OS 原理中重要但实验中未对应的知识点

1.内存保护与访问控制

OS 通过页表和权限位来实现内存保护，在本次实验中只是简单地管理空闲块，没有进程/用户态的概念，也没有权限控制。

2.交换与页置换算法

当内存不足时，通过磁盘交换、LRU、FIFO等算法回收物理页，本次实验中假设物理内存是固定的，也没有涉及磁盘交换机制。